

Does the skill make the agent better — and where?

Complete three-layer metric report across every trajectory of the latest run for each task, valid or invalid. L1 asks *did it win*, L2 asks *where in the pipeline*, L3 asks *at what cost*.

gsm8k · boolq **mvp-032** samsun **spike-018** agent **MLEvolve** model **deepseek-v4-pro** source **held_out_score.json + report.json (PVC)** generated **2026-07-01**

◆ Verdict

paired same-seed Lift = with – without



Only same-seed pairs where *both* cells produced a valid held-out score count toward Lift. **valid** = held-out grader scored a real submission; **invalid** = no gradeable artifact. The last job to finish, gsm8k-with-skill-s0, came back invalid and is included below.

■ What each layer measures

every metric the analyzer computes

Taxonomy per docs/eval/stage2.md. Boundary: L2 is per-stage attribution, L3 is trajectory aggregates. Derivations live in src/mleval/analyzer/.

L1 Outcome — did the skill solve the task better?

paired with – without, mean + 95% CI

best_metric	Held-out grader score on withheld references — the trustworthy A/B outcome. grader/*, aggregate
self_reported_metric	MLEvolve's internal stdout scalar that drives its tree search. Gameable; kept only for drift diagnostics. journal metric.value
held_out_valid	Whether the grader scored a real submission (else the outcome is None). held_out_score.json
cell mean	Mean best_metric per cell (with_skill / without_skill), per task.

paired Lift	(with – without) per (task, seed) where both are valid. <code>aggregate.paired_lifts</code>
lift_mean / stdev / CI95	Mean Lift across pairs, sample stdev, $1.96 \cdot \sigma / \sqrt{n}$ half-width. <code>aggregate</code>
cost_normalized_lift	Lift per 1k with-skill tokens and per with-skill \$ — counters 'win by spending more'. <code>metrics.cost_normalized_lift</code>

L2a Sub-stage activity — where in the pipeline

6 top-level × 16 sub-stages, multi-label

stage_counts	Node count per MCGS search stage: root · draft · debug · improve · evolution. <code>trajectory.stage</code>
sub_stage_counts	Node touches per pipeline sub-stage (1a...6c) via AST imports/calls, multi-label. <code>stage_classifier.all_sub_stages</code>
stage_chi_square	χ^2 test of the sub-stage activity distribution, with_skill vs without_skill (χ^2 , dof, p, n_stages). <code>metrics.stage_chi_square</code>
clean_reach	Of nodes touching stage s, fraction that ran clean: clean(s)/touches(s). 'Gets it right.' <code>stage_metrics</code>
rework / rework_edges	Re-attempts of stage s beyond the first: touches–1 (and the parent-edge self-loop form). 'Thrash.' <code>stage_metrics</code>
fail_modes	Distribution of exc_type over buggy nodes touching stage s. 'Which errors live here.' <code>stage_metrics</code>

L2b State predicates & process quality

free, no LLM

state predicates (4)	has_best_solution · at_least_one_non_buggy_node · best_metric_finite · prompts_log_present (+ per-task). <code>state_predicates</code>
redundant_loops	Debug edges where child AND parent are both buggy — patching broken code that stays broken. <code>metrics.redundant_loops</code>
self_correction_rate	Of debug attempts (parent buggy), fraction yielding a clean child. <code>metrics.self_correction_rate</code>
hallucination	Fraction of buggy nodes whose exc is Import/Module/Name/Attribute error — invented symbols. <code>metrics.hallucination_rate</code>
first_valid_submission	Earliest step with a valid submission (trajectory.is_valid). <code>metrics.time_to_first_valid_submission</code>
convergence	Per-step best-metric-so-far curve (+ maximize flag). <code>metrics.convergence_curve</code>
skill_api_adoption	Fraction of code-gen steps using ≥1 skill-recommended API (importsucalls). with_skill only. <code>metrics.skill_api_adoption</code>

L2c Evidence-grounded LLM judge

DEFAULT OFF — not run

--	--

choice-quality judge

Agent-as-a-Judge over per-stage evidence. Enabled only if L2a+L2b fail to discriminate cells (~3M tokens). Not computed for these runs.

L3 Trajectory cost & effort

aggregates only

cost_usd	Σ tokens × per-model OpenRouter price. <code>metrics.cost_usd + pricing.py</code>
input_tokens / output_tokens	Summed prompt / completion tokens across all LLM calls. <code>trajectory llm_total*_tokens</code>
llm_call_count	Number of LLM calls logged. <code>prompts.jsonl</code>
llm_latency	Per-call wall time p50 / p95 / max (seconds). <code>metrics.llm_latency</code>
step_exec_time	Interpreter node exec time p50 / p95 / total (seconds, excludes LLM). <code>metrics.step_exec_time</code>
step_count	Journal nodes = code-gen/exec steps (path length). <code>metrics.step_count</code>
wall_clock_sec	Pod wall clock from manifest timestamps. <code>manifest</code>
error_nodes	Count of buggy nodes in the trajectory.

L1 Outcome

did the skill solve it better?

Sweep-level outcome — per task							
Task	metric	valid pairs	with mean	without mean	mean Lift	Lift / \$ (with)	χ² p (L2a)
boolq	accuracy	2	0.874	0.810	+0.064	−0.445	0.010
gsm8k	exact_match	1	—	—	−0.334	−0.445	0.010
samsum	rougeL_f	1	0.348	0.227	+0.180	+0.809	0.935

Per-trajectory outcome (L1)							
Trajectory	cell	seed	status	held-out	self-rep	valid	Δ vs pair
BOOLQ — ACCURACY							
bq-w0	with-skill	s0	completed	0.8679	0.871	valid	+0.131

Trajectory		cell	seed	status	held-out	self-rep	valid	Δ vs pair
bq-w1		with-skill	s1	completed	0.8798	0.897	valid	-0.004
bq-o0		without-skill	s0	completed	0.7364	0.746	valid	-0.131
bq-o1		without-skill	s1	completed	0.8835	0.901	valid	+0.004
GSM8K — EXACT_MATCH								
gs-w0	last	with-skill	s0	completed	—	—	invalid	—
gs-w1	recovered	with-skill	s1	crashed	0.6634	0.736	valid	—
gs-w2		with-skill	s2	completed	0.1698	0.196	valid	-0.334
gs-o0		without-skill	s0	completed	0.6619	0.784	valid	—
gs-o1		without-skill	s1	completed	—	—	invalid	—
gs-o2		without-skill	s2	completed	0.5034	0.560	valid	+0.334
SAMSUM — ROUGEL_F								
sm-w0	backfill	with-skill	s0	crashed	0.4075	0.417	valid	+0.180
sm-w1		with-skill	s1	completed	0.2877	0.301	valid	—
sm-o0		without-skill	s0	completed	0.2271	0.233	valid	-0.180
sm-o1	backfill	without-skill	s1	crashed	—	—	invalid	—

best_metric is the trustworthy held-out grader score. **self-reported** is MLEvolve's internal stdout scalar (drives its search, gameable) — a wide gap flags off-task drift (e.g. gsm8k-without-s0: self 0.784 vs held 0.662).

L2 Stage attribution

where in the 6×16 pipeline?

L2a Stage-distribution shift (x²)

over the 16 sub-stage activity counts

Stage-distribution shift — χ² over 16 sub-stage counts (with vs without)							
Run	scope	χ²	dof	p (approx)	n stages	Lift / 1k tok	Lift / \$
mvp-032	boolq + gsm8k	18.531	7	0.00979 ✓	8	-0.0002	-0.445
spike-018	samsum	2.387	7	0.93534	8	+0.0003	+0.809

L2a Sub-stage activity — node touches

sub_stage_counts · how many nodes touched each stage

Sub-stage activity — node touches per stage										
Trajectory	1a	2b	3b	3c	4a	4b	6a	6b	6c	Σ
B00LQ										
with-skill s0	.	.	3	6	3	6	.	6	6	30
with-skill s1	.	.	.	3	3	3	3	3	3	18
without-skill s0	.	1	.	10	10	10	.	5	10	46
without-skill s1	.	12	.	11	12	12	.	12	12	71
GSM8K										
with-skill s0	.	.	.	5	3	5	.	5	5	23
with-skill s1	.	.	.	3	3	3	.	3	3	15
with-skill s2	.	.	.	2	2	2	.	2	2	10
without-skill s0	.	.	.	5	3	5	.	5	5	23
without-skill s1	.	.	.	3	4	4	.	4	4	19
without-skill s2	.	.	.	6	.	6	.	6	6	24
SAMSUM										
with-skill s0	3	.	.	3	3	3	3	3	3	21
with-skill s1	17	.	3	17	17	17	17	17	17	122
without-skill s0	15	.	.	15	15	15	15	15	15	105
without-skill s1	1	.	.	1	1	1	.	1	1	6

L2a Clean-reach — clean nodes / nodes touching stage

where the agent gets each stage right

Clean-reach — clean nodes / nodes touching stage (colour = rate)										
Trajectory	1a	2b	3b	3c	4a	4b	6a	6b	6c	
B00LQ										
with-skill s0	.	.	0.00 0/3	0.33 2/6	0.00 0/3	0.33 2/6	.	0.33 2/6	0.33 2/6	

Trajectory	1a	2b	3b	3c	4a	4b	6a	6b	6c
with-skill s1	.	.	.	0.67 2/3	0.67 2/3	0.67 2/3	0.67 2/3	0.67 2/3	0.67 2/3
without-skill s0	.	0.00 0/1	.	0.70 7/10	0.70 7/10	0.70 7/10	.	0.60 3/5	0.70 7/10
without-skill s1	.	0.75 9/12	.	0.73 8/11	0.75 9/12	0.75 9/12	.	0.75 9/12	0.75 9/12
GSM8K									
with-skill s0	.	.	.	0.00 0/5	0.00 0/3	0.00 0/5	.	0.00 0/5	0.00 0/5
with-skill s1	.	.	.	0.67 2/3	0.67 2/3	0.67 2/3	.	0.67 2/3	0.67 2/3
with-skill s2	.	.	.	0.50 1/2	0.50 1/2	0.50 1/2	.	0.50 1/2	0.50 1/2
without-skill s0	.	.	.	0.20 1/5	0.00 0/3	0.20 1/5	.	0.20 1/5	0.20 1/5
without-skill s1	.	.	.	0.00 0/3	0.00 0/4	0.00 0/4	.	0.00 0/4	0.00 0/4
without-skill s2	.	.	.	0.50 3/6	.	0.50 3/6	.	0.50 3/6	0.50 3/6
SAMSUM									
with-skill s0	0.00 0/3	.	.	0.00 0/3	0.00 0/3	0.00 0/3	0.00 0/3	0.00 0/3	0.00 0/3
with-skill s1	0.41 7/17	.	0.33 1/3	0.41 7/17	0.41 7/17	0.41 7/17	0.41 7/17	0.41 7/17	0.41 7/17
without-skill s0	0.13 2/15	.	.	0.13 2/15	0.13 2/15	0.13 2/15	0.13 2/15	0.13 2/15	0.13 2/15
without-skill s1	0.00 0/1	.	.	0.00 0/1	0.00 0/1	0.00 0/1	.	0.00 0/1	0.00 0/1

L2b State predicates & process quality

4 generic predicates + trajectory-level process derivations

State predicates & process quality (L2b)										
Trajectory	cell	seed	preds	steps	err	redund	self-fix	hallu	1st sub@	failure modes

Trajectory	cell	seed	cost	tok in	tok out	calls	lat p50	lat p95	lat max	exec p50	exec total	steps	wall s
bq-w0	with-skill	s0	\$0.0351	80,240	3,824	39	19.6	122.5	197.9	5,686	54,702	7	64,801
bq-w1	with-skill	s1	\$0.0182	34,564	1,224	23	15.0	135.1	260.8	17,171	48,599	4	64,801
bq-o0	without-skill	s0	\$0.0871	124,333	43,193	53	37.7	206.9	912.0	3,974	41,916	11	64,802
bq-o1	without-skill	s1	\$0.0584	149,612	9,752	64	28.0	196.1	465.6	5,446	57,424	13	64,801
GSM8K — EXACT_MATCH													
gs-w0	with-skill	s0	\$0.0661	66,295	35,920	34	31.5	576.6	922.9	1,742	45,106	6	64,801
gs-w1	with-skill	s1	\$0.0210	33,138	2,444	32	29.1	132.3	226.3	15,379	46,665	4	2,874
gs-w2	with-skill	s2	\$0.0139	17,561	1,265	17	39.7	122.8	123.7	19,973	39,946	3	64,805
gs-o0	without-skill	s0	\$0.0579	53,391	1,870	30	30.8	377.1	499.7	7,305	47,035	6	64,801
gs-o1	without-skill	s1	\$0.0180	37,397	1,293	25	48.0	153.9	303.0	15,210	53,929	5	64,801
gs-o2	without-skill	s2	\$0.0749	74,505	3,307	32	40.3	273.4	494.5	10,140	56,247	7	64,802
SAMSUM — ROUGEL_F													
sm-w0	with-skill	s0	\$0.1105	279,884	26,573	123	19.9	112.3	186.5	1,709	17,949	13	—
sm-w1	with-skill	s1	\$0.1123	286,044	28,484	109	15.6	141.9	326.7	994	17,280	21	—
sm-o0	without-skill	s0	\$0.1336	329,338	37,513	115	22.4	155.0	433.3	33	9,246	21	—
sm-o1	without-skill	s1	\$0.0647	146,491	17,999	59	19.8	149.1	234.7	24	3,657	7	—

Tokens/cost from prompts.jsonl × pricing.py (undercounts backoff retries by O(few %)). LLM-latency = per-call wall time; step-exec = interpreter node exec time (excludes LLM). samsum wall-clock is null (backfilled manifests carry no timestamps).

▸ Reading notes & caveats

- **Pooled mvp-032 Lift mixes metrics.** The tool averages boolq (accuracy) and gsm8k (exact_match) pairs into one number (−0.069). Read per-task: boolq +0.064 mean, gsm8k −0.334 (n=1). The cards above are per-task.
- **Unbalanced valid seeds.** gsm8k with-skill valid on {s1,s2}, without-skill valid on {s0,s2} — only s2 pairs. boolq is the only fully-paired task (both seeds, both cells).
- **Two samsum trajectories were backfilled post-hoc** (with-s0, without-s1) — they never ran finalize, so their analyzer artifacts were regenerated from the surviving journal; status shows

crashed.

- **gsm8k-with-skill-s1** is the 17h run recovered after an OOM+restart (status crashed, wall 2874s reflects the truncated restart, not the graded 17h work).

14 trajectories · 3 tasks · MLEvolve @ deepseek-v4-pro · metrics: mlevel.analyzer {metrics · stage_metrics · stage_classifier · state_predicates · aggregate} + mlevel.grader
Numbers read from persisted held_out_score.json + report.json on the mlevel-results PVC (ecepzie). Two samsum trajectories backfilled from surviving journals. L2c judge default-OFF.